

과제 #3: Marabou를 이용한 신경망 검증

2023540013 남이정

1. Marabou Resources 디렉토리 (Problem 1)

resources directory에는 Marabou를 처음 사용할 때 참고할 수 있는 여러 예제 파일들이 들어 있었다. 대표적으로 nnet, onnx, keras, properties와 같은 폴더가 있었다. nnet 폴더에는 ACAS XU, MNIST, CollisionAvoidance와 같은 benchmark network들이 포함되어 있었고, onnx 폴더에는 ONNX 형식의 신경망 모델 예제들이 있었다. keras 폴더에는 sigmoid 또는 CNN 구조의 Keras 모델 예제가 있었으며, 이를 통해 Marabou가 여러 형식의 neural network를 다룰 수 있다는 것을 확인할 수 있었다.

또한 properties 폴더에는 network가 만족해야 하는 입출력 조건을 담은 property 파일들이 들어 있었다. 예를 들어 ACAS XU 모델에 대한 property 파일들이 있었고, MNIST 관련 property 예제도 확인할 수 있었다. 이를 통해 Marabou에서는 단순히 모델 파일만 실행하는 것이 아니라, 모델과 함께 입력 조건 및 출력 조건을 정의한 property를 같이 사용하는 점을 알 수 있었다.

이 내용을 바탕으로 문제2에서는 resources directory에 이미 포함된 모델을 그대로 사용하지 않고, 직접 학습한 작은 MNIST 모델을 ONNX 형식으로 변환하여 Marabou로 검증하기로 하였다. ONNX 형식은 PyTorch에서 가져온 외부 모델에 가장 안정적인 경로로서 Marabou에서 사용하기 적절하고 MNIST는 입력 크기가 작아 verification 실험을 하기에 부담이 적다고 판단하였다.

2. 모델, 데이터셋, 검증 query

모델: 작은 fully-connected MNIST 분류기. 구조는 Flatten(784) → Linear(128) → ReLU → Linear(64) → ReLU → Linear(10)이며, Adam(lr=1e-3) + cross-entropy로 MNIST 학습 셋에서 3 epoch 학습하여 테스트 정확도 96.59%를 달성하였다. ReLU만 사용한 이유는 Marabou가 piecewise-linear 활성화함수를 native하게 지원하기 때문이며, 약 11만 파라미터 규모는 Marabou의 실용 범위 안에 들어온다. PyTorch에서 ONNX(opset 12)로 export하여 maraboupy.read_onnx로 로드하였다. 데이터셋: MNIST 테스트 셋에서 올바르게 분류된 첫 5장을 검증 anchor로 사용. 픽셀값은 [0,1] 범위를 유지하여 perturbation 반경 ϵ 이 픽셀 강도의 비율로 직관적으로 해석되도록 하였다.

검증 query: 각 (입력 x , 정답 y , 반경 ϵ)에 대해 " $\|x' - x\|_\infty \leq \epsilon$ 이면 $\text{argmax } f(x') = y$ " 성질을 검증. Marabou는 SAT solver이므로 부정형을 푼다 ; L_∞ ball 내부에 있는 x' 중 어떤 잘못된 클래스 j 에 대해 $\text{logit}_j(x') \geq \text{logit}_y(x')$ 인 x' 를 탐색한다.

9개의 잘못된 클래스에 대한 disjunction이며, maraboupy의 addDisjunctionConstraint로 인코딩하였다.

SAT은 counterexample 발견(robust 아님), UNSAT은 해당 ϵ 에서 robust함의 증명이다.

3. 결과와 해석

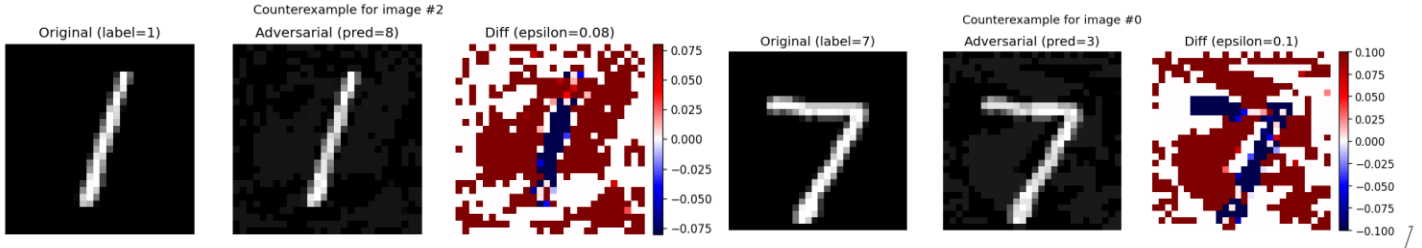
3개 이미지 × 6개 ϵ 값, query당 100초 timeout으로 실행한 결과는 다음과 같다.

이미지	정답	$\epsilon=0.001$	$\epsilon=0.005$	$\epsilon=0.01$	$\epsilon=0.02$	$\epsilon=0.08$	$\epsilon=0.1$
#0	7	UNSAT 0.14s	UNSAT 0.53s	UNSAT 0.49s	UNSAT 0.46s	TIMEOUT	SAT→3 8.15s
#1	2	UNSAT 0.16s	UNSAT 0.54s	UNSAT 0.57s	UNSAT 0.55s	TIMEOUT	TIMEOUT
#2	1	UNSAT 0.22s	UNSAT 0.81s	UNSAT 1.21s	TIMEOUT	SAT→8 6.74s	SAT→8 7.41s

주요 관찰:

- $\epsilon \leq 0.01$ 에서는 세 이미지 모두 1 초 미만에 UNSAT — 작은 perturbation에 대한 robustness가 형식적으로 증명되었다.

- Robustness 경계는 입력별로 다르다. #0, #1 은 $\epsilon=0.02$ 까지 UNSAT 인 반면 #2 는 $\epsilon=0.02$ 에서 이미 timeout. 같은 모델이라도 입력 위치에 따라 안전 마진이 다르다.
- TIMEOUT 은 항상 SAT/UNSAT 경계 근처에서 발생했다(#0 의 $\epsilon=0.08$, #2 의 $\epsilon=0.02$). 경계에서 멀어지면 Marabou 는 다시 수 초 안에 답을 낸다 — verifier 가 가장 힘들어하는 영역이 바로 결정 경계 근처라는 것을 보여준다.
- Counterexample 이 의미론적 구조를 가진다. $\epsilon=0.1$ 에서 #0(7)은 3 으로, #2(1)는 8 로 오분류 — 두 쌍 모두 시각적으로 유사한 숫자이다. Marabou 가 찾은 perturbation 은 임의의 노이즈가 아니라 인근 클래스 방향이다.
- 예상 외 사례: #1(숫자 2)은 $\epsilon=0.1$ 이라는 큰 반경에서도 counterexample 을 찾지 못하고 timeout. 결정 경계가 실제로 멀거나 Sol/DeepPoly heuristic 이 이 입력에서 잘 작동하지 않는 것으로 보인다



4. Marabou 의 강점과 한계 (실험 경험 기반)

강점:

- Soundness — UNSAT 은 robustness 의 수학적 증명이다. PGD 나 random sampling 같은 경험적 평가로는 결코 얻을 수 없는 보장.
- maraboupy 사용 편의성 — ONNX 모델은 read_onnx 한 줄로 로드되고, 본 과제의 전체 검증 루프를 Python 100 줄 미만으로 작성할 수 있었다.
- Disjunction native 지원 — top-1 robustness 는 본질적으로 잘못된 클래스에 대한 disjunction 이며 addDisjunctionConstraint 가 이를 직접 표현한다.

한계:

- Scalability — 약 11 만 파라미터의 작은 FC 네트워크에서도 경계 근처 query 는 100 초 timeout 에 자주 걸렸다. Gurobi/병렬화 없이는 이보다 큰 모델은 사실상 어려움.
- 설치/포맷 친화성 — Python 3.11 까지만 wheel 이 제공되어 conda 로 별도 환경 구성이 필요했고, 최신 PyTorch 의 dynamo ONNX exporter 는 호환되지 않아 legacy TorchScript exporter(dynamo=False)와 opset 12 를 명시해야 했다. TensorFlow 를 쓰려면 별도 설치 필요.
- TIMEOUT 시 정보 없음 — 시간 초과가 나면 “robust”인지 “not robust”인지 어느 쪽도 알 수 없다. Abstract interpretation 기반 도구라면 최소한 상하한 추정이라도 제공했을 것이다.

결론: SMT 기반 검증은 soundness와 scalability 사이의 명확한 trade-off에 위치한다. Marabou는 단독 도구로 모든 문제를 풀기보다는, α, β -CROWN 등 abstract interpretation 방식과 함께 사용되는 verification toolbox의 한 축으로 가장 효과적으로 활용될 수 있다.